SuperDrive

by Tom Smith for Nexstar Media Inc.
Current Version: 1.0.3
For Use With Ross Overdrive

Introduction

SuperDrive is a program written in Python 3.11.5 that assists directors and technical directors by automatically playing through lower third banners (supers) in an Overdrive environment in a logical fashion. Because Overdrive considers supers to be timeline events, the operator must cue and play them in order to bring them to air. The trouble with this is that it can lead to the operator either becoming behind because they forgot to advance to the new story super, or habitually double punching when no super is available and getting ahead of script. Unfortunately, Ross offers no native solution to allow the automatic playing of supers. SuperDrive works by constantly monitoring what's in program and preview. If a super is in preview but not in program, it emulates a key press to advance the timeline. Because this logic is implemented, the first super on any given line will always play up, but further supers on the line will be played through by the operator.

Setup

Because it works by emulating a keyboard press, SuperDrive needs to be run on the Overdrive client machine.There is no install wizard. Simply copy the program folder to the hard drive and launch the program.

SuperDrive has the following configurable options:

Option	Description	Default
Overdrive IP	IP Address of Active Overdrive server	10.10.78.10
Overdrive TCP Port	Port being used by Overdrive server for FloorDirectorAPI, configurable in Overdrive server	8760
SuperDrive TCP Port	Port used by SuperDrive to listen for commands from OverDrive CC's	8888
Super Keyword	The name given to the template used by supers in Overdrive, used by SuperDrive to determine whether or not something is a super	XPN 1 MOS CG

Option	Description	Default
Advance Key	The keyboard hot key used to advance the rundown in RundownControl on the Overdrive client.	space
Advance Delay (Seconds)	Time between seeing the super and the space bar press being sent. This is meant to avoid an OverDrive transition error.	0.5
Logging	Level of logging done by SuperDrive.	Info

These settings[1] are saved when changed or when SuperDrive closes to settings.pkl. They'll be automatically loaded on launch. If the file is deleted, a fresh one will be built on the next launch.

Usage

If properly configured, SuperDrive only needs to be open and running in the background for the supers to auto advance. A status bar at the bottom of the window indicates whether or not SuperDrive is active and if a super is being played. If necessary, the program can be deactivated from the File menu and reactivated as appropriate.

SuperDrive has a simple user interface that consists of two tables displaying information as it arrives in preview and program. The bottom of the window holds a status bar which shows if the program is active and if a super is being played. The UI exists mainly as a confidence feed for the operator to confirm that data is being actively received.

[1] Although the setting is present, the SuperDrive port feature is not currently used. The feature will be enabled in a future version.

Underlying Processes

SuperDrive works by piggybacking on Ross' FloorDirectorAPI, the documentation for which is at the end of this document. This API is intended for providing the floor director or other crew members with a display of what's in program and what's in preview. SuperDrive connects to the API via TCP/IP and continually requests the latest data. If the data has changed since the previous request, SuperDrive then parses the data to determine if program/preview are supers. An item is deemed to be a super if the template of the item matches the specified Super Keyword. For this reason, SuperDrive will only work in an environment where all supers utilize the same template, I.E. all supers come out of CG1. SuperDrive then employs three rules to determine whether or not a super should be played:

1. Is this super on a different line number than the last super that was automatically played?
2. Is this super on the same line number as what's in program?
3. Is what's in program currently also a super?

If the answer to all three of these questions is no, the super will be automatically played. It was crucial to limit SuperDrive to auto-playing one super per line for two reasons. Firstly, in a situation like a PKG or SOTVO, the operator needs to properly time the super reads to the speakers, so only the very first super should ever play up automatically. Secondly, a weird quirk of OverDrive/FloorDirectorAPI seems to be that it will occasionally report supers as being in preview in positions where supers do not exist; I believe that this happens when a zero M/E template is used, like CONTINUED VO, where FloorDirectorAPI briefly sees the previous item as being in prep again after the CONTINUED VO is executed.

Source Code

---

core.py (engine)

```
# -*- coding: utf-8 -*-
"""
SuperDrive Core 1.0.3
Created on Thu Aug 22 06:17:35 2024

@author: TOSmith
"""

import socket, json, keyboard, time, threading
from json_repair import repair_json

class OnAir(object):
    def __init__(self, parent, _index, slug, shot_name, template_name,
transition_name, auto_played):
        self.parent = parent
        self._index = _index
        self.slug = slug
        self.shot_name = shot_name
        self.template_name = template_name
        self.transition_name = transition_name
        self.is_super = self.super_check(self.template_name)
        self.auto_played = bool

    def __eq__(self, other):
        return self._index == other._index and self.slug == other.slug and
self.shot_name == other.shot_name and self.template_name ==
other.template_name and self.transition_name == other.transition_name and
self.is_super == other.is_super

    def __str__(self):
        return f"On Air: {self._index} \n {self.slug} \n {self.shot_name} \n
{self.template_name} \n {self.transition_name}"

    def super_check(self,template_name):
        if template_name == self.parent.super_keyword:
            return True
        return False

class Prepared(object):
    def __init__(self, parent, _index, slug, shot_name, template_name,
transition_name, auto_played):
        self.parent = parent
        self._index = _index
        self.slug = slug
        self.shot_name = shot_name
        self.template_name = template_name
        self.transition_name = transition_name
        self.is_super = self.super_check(self.template_name)
        self.auto_played = bool

    def __eq__(self, other):
```

```
        return self._index == other._index and self.slug == other.slug and
self.shot_name == other.shot_name and self.template_name ==
other.template_name and self.transition_name == other.transition_name and
self.is_super == other.is_super

    def __str__(self):
        return f"Prepared: {self._index} \n {self.slug} \n {self.shot_name}
\n {self.template_name} \n {self.transition_name}"

    def super_check(self,template_name):
        if template_name == "XPN 1 MOS CG":
            return True
        return False

class Listener(threading.Thread):
    def __init__(self, parent, connection_info: tuple, super_keyword: str,
advance_key: str):
        threading.Thread.__init__(self)
        self.parent = parent
        self.OverDrive_IP = connection_info[0]
        self.OverDrive_port = connection_info[1]
        self.request = {'protocolVersion': '1',
                                'endpoint': 'shots',
                                'correlationId': '0'
                                }
        self.listening = False
        self.previous_data = None
        self.super_keyword = super_keyword
        self.advance_key = advance_key
        self.prepared = None
        self.on_air = None

    def validate_data(self, data):
        decoded = data.decode('UTF-8')
        try:
            jsonified = json.loads(decoded)
            if 'shots' in jsonified:
                return (True, jsonified)
            elif 'cues' in jsonified:
                return (False, None)
            return (False, None)
        except json.JSONDecodeError:
            self.parent.log.error(f'JSON Decode error: {decoded} \n Trying to
repair.')
            try:
                jsonified = repair_json(decoded)
                jsonified = json.loads(jsonified)
                if 'shots' in jsonified:
                    return (True, jsonified)
                elif 'cues' in jsonified:
                    return (False, None)
                return (False, None)
            except Exception as e:
                self.parent.log.error(f"Couldn't repair JSON: {e}.
Discarding.")
            return(False, None)

    def parse_data(self, data):
```

```
        prepared = data['shots'][1]
        on_air = data['shots'][0]
        prepared = Prepared(self,
                            _index = prepared['index'],
                            slug = prepared['slug'],
                            shot_name = prepared['shotName'],
                            template_name = prepared['templateName'],
                            transition_name = prepared['transitionName'],
                            auto_played = bool)
        on_air = OnAir(self,
                       _index = on_air['index'],
                       slug = on_air['slug'],
                       shot_name = on_air['shotName'],
                       template_name = on_air['templateName'],
                       transition_name = on_air['transitionName'],
                       auto_played = bool)
        return prepared, on_air

    def present_data(self, data):
        prepared = data[0]
        prepared_changed = False
        on_air = data[1]
        on_air_changed = False
        if self.first_pass:
            #Add the items to the lists for the OLV's
            self.prepared_olv.append(prepared)
            self.on_air_olv.append(on_air)

            #Set the mutables
            self.prepared = prepared
            self.on_air = on_air
            self.last_check = [prepared, on_air]

            #Add them to the log
            self.parent.log.info(prepared)
            self.parent.log.info(on_air)

            #Set the OLV's
            self.parent.olv_preview.SetObjects(self.prepared_olv)
            self.parent.olv_program.SetObjects(self.on_air_olv)

            #Set the first pass flag
            self.first_pass = False
        else:
            #Check if prepared or on air have changed
            if prepared != self.prepared:
                #Add the item to the list for the OLV
                self.prepared_olv.append(prepared)

                #Set the mutables
                self.prepared = prepared
                self.last_check[0] = prepared
                prepared_changed = True

                #Add to the log
                self.parent.log.info(prepared)

                #Refresh the OLV
```

```
                self.parent.olv_preview.SetObjects(self.prepared_olv)
                print(self.prepared)
            if on_air != self.on_air:
                #Add the item to the list for the OLV
                self.on_air_olv.append(on_air)

                #Set the mutables
                self.on_air = on_air
                self.last_check[1] = on_air
                on_air_changed = True

                #Add to the log
                self.parent.log.info(on_air)

                #Refresh the OLV
                self.parent.olv_program.SetObjects(self.on_air_olv)
                print(self.on_air)
            if prepared_changed or on_air_changed:
                print('Handling super')
                self.handle_super(prepared,on_air)


    def handle_super(self, prepared, on_air):
        try:
            if prepared._index == on_air._index:
                print('Index match.')
                if prepared.is_super == True and on_air.is_super != True and
self.last_super_index != prepared._index:
                    time.sleep(self.parent.press_delay)
                    keyboard.press_and_release(self.parent.advance_key)
                    self.last_super_index = prepared._index
        except AttributeError as e:
            print(f'Error advancing super: {e}')
            self.parent.log.error(e)

    def run(self):
        self.listening = True
        self.parent.statusbar.SetStatusText("SuperDrive is active.", 0)
        self.prepared_olv = []
        self.on_air_olv = []
        self.last_check = []
        self.prepared = None
        self.on_air = None
        self.last_super_index = None
        self.first_pass = True
        self.parent.set_columns()
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
            try:
                #Turn off timer updates to reduce network traffic.
                s.connect((self.OverDrive_IP, self.OverDrive_port))
                request = {'protocolVersion': '1',
                           'endpoint': 'subscription',
                           'reqData': 'type=optOut',
                           'correlationId': '1000001'
                           }
                s.sendall(bytearray(json.dumps(request), 'UTF-8'))
                data = s.recv(1026)
```

```
            while self.listening:
                s.sendall(bytearray(json.dumps(self.request), 'UTF-8'))

                # Listen for incoming data
                data = s.recv(1026)

                if not data:
                    print("Connection closed by the server.")
                    break  # Exit the loop if the connection is closed

                else:
                    validation = self.validate_data(data)
                    if validation[0]:
                        try:
                            parsed = self.parse_data(validation[1])
                            self.present_data(parsed)
                        except TypeError as e:
                            print(f"Type error in data: {e}")
                            self.parent.log.error(e)

        except socket.error as e:
            print(f"Socket error: {e}")

            s.close()
            print('Socket closed.')

        self.parent.statusbar.SetStatusText("SuperDrive is not active.",
0)
```

main.pyw (user interface)

```
# -*- coding: utf-8 -*-
"""
SuperDrive GUI
Created on Thu Aug 22 08:15:40 2024

@author: TOSmith
"""

import wx #GUI
from core import Listener #Backend with FloorDirectorAPI
from ObjectListView import FastObjectListView, ObjectListView, ColumnDefn
#Data Presentation
import keyboard, pickle, os #Stroke capture, data persistence, file checking
import webbrowser #Open documentation
import logging

class MainFrame(wx.Frame):
    def
__init__(self,overdrive_connection,superdrive_port,super_keyword,advance_key,
press_delay, log, log_level):
        super().__init__(parent=None,title="SuperDrive")
        self.title = "SuperDrive 1.0.3"
        self.SetTitle(self.title)
        self.SetIcon(wx.Icon('img/icon.ico'))
        self.overdrive_connection = overdrive_connection
        self.superdrive_port = superdrive_port
        self.super_keyword = super_keyword
        self.advance_key = advance_key
        self.overdrive = Listener(self, self.overdrive_connection,
self.super_keyword, self.advance_key)
        self.press_delay = press_delay #Seconds to wait before triggering the
take and prepare.
        self.log = log
        self.log_level = log_level
        logging.basicConfig(filename='SuperDrive.log',level=self.log_level)
        #self.receiver = Receiver(self)
        self.Bind(wx.EVT_CLOSE,self.on_close)
        '''Init Layout'''
        self.panel_main = wx.Panel(self)
        self.sizer_main = wx.FlexGridSizer(2,2,0,0)
        self.sizer_main.AddGrowableRow(1,1)
        self.sizer_main.AddGrowableCol(0,1)
        self.sizer_main.AddGrowableCol(1,1)
        self.statusbar = self.CreateStatusBar(2)

        '''Control Widgets'''
        self.label_control = wx.StaticText(self.panel_main,label="")

        '''Preview Widgets'''
        self.label_preview = wx.StaticText(self.panel_main, label="Prepared")
        self.olv_preview = FastObjectListView(self.panel_main, wx.ID_ANY,
style=wx.LC_REPORT|wx.SUNKEN_BORDER|wx.EXPAND)
        self.olv_preview.cellEditMode = FastObjectListView.CELLEDIT_NONE
        self.olv_preview.Bind(wx.EVT_CHAR_HOOK, self.on_key_down)
```

```
        '''Program Widgets'''
        self.label_program = wx.StaticText(self.panel_main, label="On Air")
        self.olv_program = FastObjectListView(self.panel_main, wx.ID_ANY,
style=wx.LC_REPORT|wx.SUNKEN_BORDER|wx.EXPAND)
        self.olv_program.cellEditMode = FastObjectListView.CELLEDIT_NONE
        self.olv_program.Bind(wx.EVT_CHAR_HOOK, self.on_key_down)

        '''Filling Sizers'''
        self.sizer_main.AddMany([(self.label_preview,1,wx.ALL|wx.CENTER),
                                 (self.label_program,1,wx.ALL|wx.CENTER),
                                 (self.olv_preview,1,wx.ALL|wx.EXPAND),
                                 (self.olv_program,1,wx.ALL|wx.EXPAND)])
        self.panel_main.SetSizerAndFit(self.sizer_main)
        self.SetInitialSize(self.GetBestSize())
        self.SetMinSize((450,100))
        self.Layout()
        self.overdrive.start()
        #self.receiver.start()
        self.create_menu()
        self.Show()

    def on_key_down(self, event):
        keycode = event.GetKeyCode()
        if keycode == wx.WXK_SPACE:
            pass
        else:
            event.Skip()

    def set_columns(self):
        self.col_index = ColumnDefn("Page", "left", -1, "_index",
isSpaceFilling=True)
        self.col_slug = ColumnDefn("Slug", "left", -1, "slug",
isSpaceFilling=True)
        self.col_shot = ColumnDefn("Shot", "left", -1, "shot_name",
isSpaceFilling=True)
        self.col_template = ColumnDefn("Template", "left", -1,
"template_name", isSpaceFilling=True)
        self.col_transition = ColumnDefn("Transition", "left", -1,
"transition_name", isSpaceFilling=True)
        self.col_is_super = ColumnDefn("Super", "left", -1, "is_super",
isSpaceFilling=True)
        self.olv_preview.SetColumns([self.col_index,
                                     self.col_slug,
                                     self.col_shot,
                                     self.col_template,
                                     self.col_transition,
                                     self.col_is_super])
        self.olv_program.SetColumns([self.col_index,
                                     self.col_slug,
                                     self.col_shot,
                                     self.col_template,
                                     self.col_transition,
                                     self.col_is_super])
        try:
            self.olv_preview.SetObjects(self.overdrive.prepared_olv)
        except Exception as e:
            print(e)
        try:
```

```
            self.olv_program.SetObjects(self.overdrive.on_air_olv)
        except Exception as e:
            print(e)

    def on_close(self, event):
        dlg = wx.MessageDialog(self,'Are you sure you want to quit
SuperDrive?','Quit?',style=wx.YES_NO)
        result = dlg.ShowModal()
        if result == wx.ID_YES:
            self.overdrive.listening = False
            if hasattr(self, 'receiver'):
                self.receiver.receiving = False
            with open('settings.pkl','wb') as file:
                settings = [self.overdrive_connection,
                            self.superdrive_port,
                            self.super_keyword,
                            self.advance_key,
                            self.press_delay,
                            self.log_level]
                pickle.dump(settings,file)
                file.close()
            self.Destroy()

    def create_menu(self):
        menu_bar = wx.MenuBar()

        file_menu = wx.Menu()
        active = file_menu.Append(wx.ID_ANY, 'Activate/Deactivate', 'Start or
stop SuperDrive.')
        self.Bind(event=wx.EVT_MENU,handler=self.on_active,source=active)
        _quit = file_menu.Append(wx.ID_ANY, 'Quit', 'Quit SuperDrive.')
        self.Bind(event=wx.EVT_MENU,handler=self.on_close,source=_quit)
        menu_bar.Append(file_menu, '&File')

        edit_menu = wx.Menu()
        settings = edit_menu.Append(wx.ID_ANY, 'Settings', 'Adjust settings')
        self.Bind(event=wx.EVT_MENU,handler=self.on_settings,source=settings)
        menu_bar.Append(edit_menu, '&Edit')

        help_menu = wx.Menu()
        about_option = help_menu.Append(wx.ID_ANY, 'About','Information about
this program.')

self.Bind(event=wx.EVT_MENU,handler=self.on_about,source=about_option)

        documentation_option = help_menu.Append(wx.ID_ANY,
'Documention',"Opens a PDF of this program's documentation.")

self.Bind(event=wx.EVT_MENU,handler=self.on_documentation,source=documentatio
n_option)

        menu_bar.Append(help_menu, '&Help')

        self.SetMenuBar(menu_bar)

    def on_active(self, event):
        if self.overdrive.listening:
            self.overdrive.listening = False
```

```
        else:
            self.overdrive = Listener(self, self.overdrive_connection,
self.super_keyword, self.advance_key)
            self.overdrive.start()

    def on_remote_start(self):
        self.overdrive.listening = False
        self.overdrive = Listener(self, self.overdrive_connection,
self.super_keyword, self.advance_key)
        self.overdrive.start()

    def on_settings(self, event):
        SettingsFrame(self)

    def on_about(self, event):
        AboutFrame(self)

    def on_documentation(self, event):
        path = os.getcwd()
        webbrowser.open(f'{path}\\documentation\\documentation.pdf')

class SettingsFrame(wx.Frame):
    def __init__(self, parent):
        super().__init__(parent=parent, title="SuperDrive Settings")
        self.parent = parent
        self.Bind(wx.EVT_CLOSE, self.on_close)

        '''Init UI'''
        self.SetTitle(f'{self.parent.title} - Settings')
        self.SetIcon(wx.Icon('img/icon.ico'))
        self.parent = parent
        self.panel_main = wx.Panel(self)
        self.sizer = wx.FlexGridSizer(8,2,10,10)
        self.sizer_buttons = wx.BoxSizer(wx.HORIZONTAL)
        self.log_levels = {"None": logging.NOTSET,
                           "Debug": logging.DEBUG,
                           "Info": logging.INFO,
                           "Warning": logging.WARNING,
                           "Error": logging.ERROR,
                           "Critical": logging.CRITICAL}
        self.log_choices = []
        for key in self.log_levels.keys():
            self.log_choices.append(key)

        '''Widgets'''
        self.label_key = wx.StaticText(self.panel_main,label="OverDrive
Advance Key")
        self.button_key = wx.ToggleButton(self.panel_main, label="Assign
Key")
        self.button_key.SetLabel(self.parent.advance_key)
        self.assigned = self.parent.advance_key
        self.button_key.Bind(wx.EVT_TOGGLEBUTTON, self.assign_key)
        self.button_key.Bind(wx.EVT_KEY_UP, self.get_key)

        self.label_overdrive_ip =
wx.StaticText(self.panel_main,label="OverDrive Server IP")
        self.field_overdrive_ip = wx.TextCtrl(self.panel_main)
        self.field_overdrive_ip.SetValue(self.parent.overdrive.OverDrive_IP)
```

```
        self.label_overdrive_port =
wx.StaticText(self.panel_main,label="OverDrive Port")
        self.field_overdrive_port = wx.TextCtrl(self.panel_main)

self.field_overdrive_port.SetValue(str(self.parent.overdrive.OverDrive_port))

        self.label_superdrive_port =
wx.StaticText(self.panel_main,label="SuperDrive Port")
        self.field_superdrive_port = wx.TextCtrl(self.panel_main)
        self.field_superdrive_port.SetValue(str(self.parent.superdrive_port))

        self.label_super_keyword = wx.StaticText(self.panel_main,label="Super
Keyword")
        self.field_super_keyword = wx.TextCtrl(self.panel_main)

self.field_super_keyword.SetValue(self.parent.overdrive.super_keyword)

        self.label_press_delay = wx.StaticText(self.panel_main,label="Advance
Delay (Seconds)")
        self.field_press_delay = wx.TextCtrl(self.panel_main)
        self.field_press_delay.SetValue(str(self.parent.press_delay))
        self.spinner_press_delay = wx.SpinButton(self.panel_main)
        self.spinner_press_delay.Bind(wx.EVT_SPIN_UP, self.spinner_up)
        self.spinner_press_delay.Bind(wx.EVT_SPIN_DOWN, self.spinner_down)
        self.sizer_press_delay = wx.BoxSizer(wx.HORIZONTAL)
        self.sizer_press_delay.AddMany([(self.field_press_delay,1,wx.ALL|
wx.CENTER),
                                        (self.spinner_press_delay,1,wx.ALL|
wx.CENTER)])

        self.label_log_level = wx.StaticText(self.panel_main,label="Logging")
        self.choice_log_level =
wx.Choice(self.panel_main,choices=self.log_choices)
        current_level = logging.getLogger().getEffectiveLevel()
        for key, value in self.log_levels.items():
            if value == current_level:
                n = list(self.log_levels.keys()).index(key)
                self.choice_log_level.SetSelection(n)

        self.button_apply = wx.Button(self.panel_main,label="Apply")
        self.button_apply.Bind(wx.EVT_BUTTON, self.on_apply)
        self.button_cancel = wx.Button(self.panel_main,label="Cancel")
        self.button_cancel.Bind(wx.EVT_BUTTON, self.on_cancel)

        '''Filling Sizers'''
        self.sizer_buttons.AddMany([(self.button_apply,1,wx.ALL|wx.CENTER),
                                    (self.button_cancel,1,wx.ALL|wx.CENTER)])
        self.sizer.AddMany([(self.label_key,1,wx.ALL|wx.CENTER),
                            (self.button_key,1,wx.ALL|wx.CENTER),
                            (self.label_overdrive_ip,1,wx.ALL|wx.CENTER),
                            (self.field_overdrive_ip,1,wx.ALL|wx.CENTER),
                            (self.label_overdrive_port,1,wx.ALL|wx.CENTER),
                            (self.field_overdrive_port,1,wx.ALL|wx.CENTER),
                            (self.label_superdrive_port,1,wx.ALL|wx.CENTER),
                            (self.field_superdrive_port,1,wx.ALL|wx.CENTER),
                            (self.label_super_keyword,1,wx.ALL|wx.CENTER),
                            (self.field_super_keyword,1,wx.ALL|wx.CENTER),
```

```
                            (self.label_press_delay,1,wx.ALL|wx.CENTER),
                            (self.sizer_press_delay,1,wx.ALL|wx.CENTER),
                            (self.label_log_level,1,wx.ALL|wx.CENTER),
                            (self.choice_log_level,1,wx.ALL|wx.CENTER),
                            (self.sizer_buttons,1,wx.ALL|wx.CENTER)])
        self.panel_main.SetSizerAndFit(self.sizer)
        self.SetInitialSize(self.GetBestSize())
        self.Layout()
        self.Show()

    def get_key(self, event):
        if self.button_key.GetValue() == True:
            self.assigned = keyboard.read_key()
            self.button_key.SetLabel(str(self.assigned))
            self.button_key.SetValue(False)
            self.panel_main.SetFocus()
            print(f'Assigned {self.assigned} as key.')

    def assign_key(self, event):
        self.panel_main.SetFocus()
        if hasattr(self, 'assigned'):
            del(self.assigned)
        self.button_key.SetValue(True)
        self.button_key.SetLabel("[[Press any key.]]")

    def spinner_down(self, event):
        current = float(self.field_press_delay.GetValue())
        current -= 0.25
        self.field_press_delay.SetValue(str(current))

    def spinner_up(self, event):
        current = float(self.field_press_delay.GetValue())
        current += 0.25
        self.field_press_delay.SetValue(str(current))

    def on_close(self, event):
        dlg = wx.MessageDialog(self,'Quit without saving your
changes?','Quit?',wx.YES_NO)
        result = dlg.ShowModal()
        if result == wx.ID_YES:
            self.Destroy()

    def on_cancel(self, event):
        self.Destroy()

    def on_apply(self, event):
        if hasattr(self, 'assigned'):
            self.parent.overdrive.advance_key = self.assigned
        overdrive_ip = self.field_overdrive_ip.GetValue()
        overdrive_port = int(self.field_overdrive_port.GetValue())
        log_level =
self.log_levels[self.choice_log_level.GetString(self.choice_log_level.GetSele
ction())]
        logging.getLogger().setLevel(log_level)
        self.parent.overdrive_connection = (overdrive_ip, overdrive_port)
        self.parent.superdrive_port =
int(self.field_superdrive_port.GetValue())
        self.parent.super_keyword = self.field_super_keyword.GetValue()
```

```
        self.parent.press_delay = float(self.field_press_delay.GetValue())
        if self.parent.overdrive.listening:
            self.parent.overdrive.listening = False
        self.parent.overdrive = Listener(self.parent,
self.parent.overdrive_connection, self.parent.super_keyword,
self.parent.advance_key)
        self.parent.overdrive.start()
        self.Destroy()

class AboutFrame(wx.Frame):
    def __init__(self, parent):
        super().__init__(parent=parent)
        self.parent = parent
        self.SetTitle(f'{self.parent.title} - About')
        self.SetIcon(wx.Icon('img/icon.ico'))
        self.panel_main = wx.Panel(self)
        self.sizer_main = wx.FlexGridSizer(6,1,10,10)
        self.font = wx.Font(12, wx.FONTFAMILY_MODERN, 0, 90, underline =
False, faceName ="Arial Bold")
        self.logo = wx.Image('img/icon.png', wx.BITMAP_TYPE_PNG)
        self.bitmap_logo =
wx.StaticBitmap(self.panel_main,bitmap=self.logo.ConvertToBitmap())
        self.label_program_name = wx.StaticText(self.panel_main,
label=self.parent.title)
        self.label_program_name.SetFont(self.font)
        self.label_byline = wx.StaticText(self.panel_main, label="by Tom
Smith for Nexstar Media Inc.")
        self.label_email = wx.StaticText(self.panel_main,
label="thomas.smith@woodtv.com")
        self.label_phone_cell = wx.StaticText(self.panel_main, label="Cell:
(231) 343.9803")
        self.label_phone_desk = wx.StaticText(self.panel_main, label="Desk:
(616) 456.8888 Ext. 4319")

        self.sizer_main.AddMany([(self.bitmap_logo,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                 (self.label_program_name,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                 (self.label_byline,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                 (self.label_email,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                 (self.label_phone_cell,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER),
                                 (self.label_phone_desk,1,wx.ALL|wx.CENTER|
wx.ALIGN_CENTER)])

        self.panel_main.SetSizerAndFit(self.sizer_main)
        self.SetInitialSize(self.GetBestSize())
        self.Layout()
        self.Show()



def main():
    logger = logging.getLogger(__name__)
    if os.path.isfile('settings.pkl'):
        with open('settings.pkl','rb') as file:
```

```
            overdrive_connection, superdrive_port, super_keyword,
advance_key, press_delay, log_level = pickle.load(file)
            file.close()
    else:
        overdrive_connection = ('10.10.78.10', 8760)
        superdrive_port = 8888
        super_keyword = "XPN 1 MOS CG"
        advance_key = "space"
        press_delay = 0.5
        log_level =  logging.INFO
    app=[]; app = wx.App(None)
    frame = MainFrame(overdrive_connection, superdrive_port, super_keyword,
advance_key, press_delay, logger, log_level)
    app.SetTopWindow(frame)
    app.MainLoop()

if __name__ == "__main__":
    main()
```